Advanced Algorithms

Klevis Imeri Lecture Notes

Autumn 2026

Contents

1 Approximation Algorithms **1**

Questions 10

2 Approximation schemes **12**

2.1 Knapsack 13

2.2 Bin packing 17

Questions 21

General questions **22**

1 Approximation Algorithms

Definition 1.1 (Problem)**.**

$$\Pi := (\mathcal{I}, \mathcal{S}).$$

$$\text{Input:} \quad I \in \mathcal{I},$$
$$\text{Output:} \quad S \in \mathcal{S}_I.$$

Π : problem,
$\mathcal{I}$: instance space, I : instance,
$\mathcal{S}$: set of possible outputs, S : output,
$\mathcal{S}_I := \{\, S \in \mathcal{S} : S \text{ is valid for } I \,\}$.

Definition 1.2 (Optimization problem)**.**

$$\Pi := (\mathcal{I}, \mathcal{S}).$$

$$c : \mathcal{I} \times \mathcal{S} \to \mathbb{R}_{\geq 0}.$$

$$\text{For fixed } I: \quad c(S) := c(I, S), \qquad c(\mathrm{OPT}(I)) := c(I, \mathrm{OPT}(I)).$$

$$\mathrm{OPT}(I) \in \mathcal{S}_I := \begin{cases} \arg\min\limits_{T \in \mathcal{S}} c(T), & \text{minimization,} \\ \arg\max\limits_{T \in \mathcal{S}} c(T), & \text{maximization.} \end{cases} + \text{ other constraints.}$$

Π : optimization problem,
$\mathcal{I}$: instance space, I : instance,
$\mathcal{S}$: set of possible outputs, S : output,
$\mathcal{S}_I$: optimal solutions for I,
c : cost function, $c(S)$: cost of S for I,
$\mathrm{OPT}(I)$: optimal solution for I (assumed to exist).

Definition 1.3 (Algorithm)**.**

$$\mathcal{A} : \mathcal{I} \to \mathcal{S}, \qquad \forall I \in \mathcal{I} : \ \mathcal{A}(I) \in \mathcal{S}_I.$$

$$\Pi := (\mathcal{I}, \mathcal{S}).$$

Π : problem,
$\mathcal{I}$: instance space, I : instance,
$\mathcal{S}$: set of possible outputs,
$\mathcal{S}_I := \{\, S \in \mathcal{S} : S \text{ is valid for } I \,\}$,
$\mathcal{A}$: algorithm, $\mathcal{A}(I)$: solution returned on I.

Definition 1.4 (Polynomial)**.**

$$\mathrm{poly}(n) := \{\, f : \exists c \in \mathbb{N}, \ f \in O(n^c) \,\}.$$

$\mathrm{poly}(n)$: functions bounded by a polynomial in n,
n : input size,
f : function,
c : constant exponent,
$O(n^c)$: asymptotic upper bound of order n^c.

Definition 1.5 (Running time (TODO))**.**

Definition 1.6 (Efficient algorithm)**.**

$$\exists p \in \mathrm{poly}(n) \ \forall I \in \mathcal{I} : \quad \mathrm{time}_{\mathcal{A}}(I) \leq p(|I|).$$

$\mathcal{A}$: algorithm,
$\mathcal{I}$: instance space, I : instance,
$|I|$: encoding length of the instance I,
$\mathrm{time}_{\mathcal{A}}(I)$: running time of $\mathcal{A}$ on I,
p : polynomial bound, n : input size.

Definition 1.7 (α-approximation)**.**

$$\mathcal{A} \text{ is an } \alpha\text{-approximation} \iff$$
$$\big[\exists p \in \text{poly}(n)\ \forall I \in \mathcal{I} : \text{time}_{\mathcal{A}}(I) \le p(|I|)\big] \wedge$$
$$\forall I \in \mathcal{I} :$$
$$\begin{cases} c\big(\mathcal{A}(I)\big) \le \alpha(I)\, c\big(\text{OPT}(I)\big), & \text{minimization,} \\ c\big(\text{OPT}(I)\big) \le \alpha(I)\, c\big(\mathcal{A}(I)\big), & \text{maximization.} \end{cases}$$

$$\Pi := (\mathcal{I}, \mathcal{S}), \qquad \alpha : \mathcal{I} \to \mathbb{R}_{\ge 1}.$$
$$\mathcal{A} : \mathcal{I} \to \mathcal{S}, \qquad \forall I \in \mathcal{I} :\ \mathcal{A}(I) \text{ is feasible for } I.$$

Π : optimization problem,
$\mathcal{I}$: instance space, I : instance,
$\mathcal{S}$: set of possible outputs,
$\mathcal{S}_I$: optimal solutions for I,
c : cost function,
$\mathcal{A}$: algorithm,
$\text{OPT}(I)$: optimal solution for I,
α : approximation-factor function,
$\alpha(I)$: approximation factor for I.

Constant factor $a \ge 1$: $\quad \alpha(I) := a \quad (\forall I \in \mathcal{I})$.

Definition 1.8 (Minimum set cover)**.**

$$\Pi_{\text{minimum set cover}} := (\mathcal{I}, \mathcal{S}).$$

$$\mathcal{I} := \left\{ (U, S, w) : 1 \leq |U| < \infty,\ S \subseteq 2^U,\ \bigcup_{S_i \in S} S_i = U,\ w : S \to \mathbb{R}_{\geq 0} \right\},$$

$$I = (U, S, w) \in \mathcal{I}, \qquad c(T) := \sum_{S_i \in T} w(S_i),$$

$$\text{OPT}(I) \in \mathcal{S}_I := \left\{ T \subseteq S : \bigcup_{S_i \in T} S_i = U \wedge\ T \in \underset{\substack{T' \subseteq S \\ \bigcup_{S_i \in T'} S_i = U}}{\arg\min}\ c(T') \right\}.$$

$\mathcal{I}$: instance space,
$\mathcal{S}$: set of possible outputs,
$\mathcal{S}_I$: minimum-cost covers of U,
c : cost function,
I : instance,
U : nonempty finite universe,
S : collection of available subsets of U,
S_i : individual available set,
T : selected subcollection,
$w(S_i)$: cost of set S_i,
$c(T)$: total cost of the selected sets,
2^U : power set of U.

Example 1.9 (Minimum set cover)**.**

$$U := \{e_1, e_2, e_3, e_4, e_5\}, \qquad S := \{S_1, S_2, S_3, S_4\}, \qquad I := (U, S, w).$$

i	S_i	$w(S_i) := i^2$
1	$\{e_1, e_2, e_5\}$	1
2	$\{e_4\}$	4
3	$\{e_2, e_3\}$	9
4	$\{e_1, e_4, e_5\}$	16

$$T := \{S_3, S_4\} \subseteq S, \qquad \bigcup_{S_i \in T} S_i = U, \qquad c(T) = 9 + 16 = 25.$$

$$T^* := \text{OPT}(I) = \{S_1, S_2, S_3\}, \qquad c(T^*) = 1 + 4 + 9 = 14.$$

$$|T^*| = 3 > 2 = |T|, \qquad c(T^*) = 14 < 25 = c(T).$$

Theorem 1.10 (Complexity of minimum set cover)**.**

$$\Pi_{\textit{minimum set cover}} \in \text{NP-hard}.$$

$$\mathrm{SC}_{\mathrm{dec}} := \left\{ (U, S, k) : \begin{array}{c} |U| < \infty,\ S \subseteq 2^U,\ k \in \mathbb{N}, \\ \exists T \subseteq S :\ |T| \leq k,\ \bigcup_{S_i \in T} S_i = U \end{array} \right\}.$$

$$\mathrm{SC}_{\mathrm{dec}} \in \mathrm{NPC}.$$

Proof (Intuition). A proposed cover can be checked in polynomial time: count its sets and check that their union is the universe. Thus the decision problem is in NP.
Vertex Cover, a known NP-complete problem, asks whether at most k vertices can touch every edge of a graph. Turn each edge into an element of U, and each vertex into the set of edges touching it. Selecting vertices that touch every edge is then equivalent to selecting sets that cover U. This construction takes polynomial time, so Set Cover is NP-hard and hence NP-complete as a decision problem.
With every set cost equal to 1, an optimal cover also tells us whether a cover with at most k sets exists. Finding an optimal cover is therefore NP-hard. □

Definition 1.11 (Greedy set cover algorithm)**.**

$$\mathcal{A}(I) := \mathsf{GreedySetCover}(U, S, w).$$

$$\mathrm{ppi}(S_j) := \begin{cases} \dfrac{w(S_j)}{|S_j|}, & S_j \neq \varnothing, \\ \infty, & S_j = \varnothing. \end{cases}$$

Algorithm 1 $\mathsf{GreedySetCover}(U, S, w)$

if $U = \varnothing$ **then**
  return $\varnothing$
else
  $S_i \in \arg\min_{S_j \in S} \mathrm{ppi}(S_j)$
  $U' \leftarrow U \setminus S_i$
  $S' \leftarrow \{\, S'_j := S_j \setminus S_i : S_j \in S \,\}$
  $w(S'_j) \leftarrow w(S_j) \quad (\forall S'_j \in S')$
  return $\{S_i\} \cup \mathsf{GreedySetCover}(U', S', w)$
end if

$$\Pi_{\text{minimum set cover}} = (\mathcal{I}, \mathcal{S}).$$

U : remaining universe,
S : collection of remaining sets,
$w(S_j)$: cost of set S_j,
$\mathrm{ppi}(S_j)$: cost per remaining element of S_j,
S_i : set chosen in the current call,
U' : universe after removing the elements of S_i,
S'_j : remaining elements of S_j,
S' : collection of sets for the recursive call.

Example 1.12 (Greedy set cover)**.**

$$U_0 := \{e_1, e_2, e_3, e_4, e_5\}, \qquad S := \{S_1, S_2, S_3, S_4\}.$$

$$S_1 = \{e_1, e_2, e_5\}, \qquad S_2 = \{e_4\},$$
$$S_3 = \{e_2, e_3\}, \qquad S_4 = \{e_1, e_4, e_5\}.$$

$$w(S_j) := j^2 \quad (j \in \{1, 2, 3, 4\}), \qquad I := (U_0, S, w).$$

$$S_j^{(t)} := S_j \cap U_t, \qquad S^{(t)} := \{\, S_j^{(t)} : j \in \{1, 2, 3, 4\} \,\}.$$

$$\mathrm{ppi}_{j,t} := \mathrm{ppi}(S_j^{(t)}) = \begin{cases} \dfrac{w(S_j)}{|S_j^{(t)}|}, & S_j^{(t)} \neq \varnothing, \\ \infty, & S_j^{(t)} = \varnothing. \end{cases}$$

$$i_t \in \operatorname*{arg\,min}_{j \in \{1,2,3,4\}} \mathrm{ppi}_{j,t}, \qquad U_{t+1} := U_t \setminus S_{i_t}.$$

t	U_t	$\mathrm{ppi}_{1,t}$	$\mathrm{ppi}_{2,t}$	$\mathrm{ppi}_{3,t}$	$\mathrm{ppi}_{4,t}$	S_{i_t}	U_{t+1}
0	U_0	$\frac{1}{3}$	4	$\frac{9}{2}$	$\frac{16}{3}$	S_1	$\{e_3, e_4\}$
1	$\{e_3, e_4\}$	∞	4	9	16	S_2	$\{e_3\}$
2	$\{e_3\}$	∞	∞	9	∞	S_3	$\varnothing$

$$\mathsf{GreedySetCover}(U_3, S^{(3)}, w) = \varnothing.$$

$$\begin{aligned} \mathcal{A}(I) &= \mathsf{GreedySetCover}(U_0, S^{(0)}, w) \\ &= \{S_1\} \cup \big(\{S_2\} \cup (\{S_3\} \cup \varnothing)\big) \\ &= \{S_1, S_2, S_3\}, \end{aligned}$$

$$c\big(\mathcal{A}(I)\big) = 1 + 4 + 9 = 14.$$

Theorem 1.13 (Termination, feasibility, and efficiency of greedy set cover)**.**

$\forall I = (U, S, w) \in \mathcal{I}:$ $\mathsf{GreedySetCover}(U, S, w)$ *terminates, is feasible, and is efficient.*

Proof.
$I = (U, S, w) \in \mathcal{I}$, $\mathcal{A}(I) := \mathsf{GreedySetCover}(U, S, w)$,
$U_t:$ remaining universe after t selections, $U_0 := U$.

1. Termination: $\mathcal{A}(I)$ terminates $\iff$ $\exists r \in \mathbb{N}_{\geq 0}:\ U_r = \varnothing$.

$i_t:$ index of the greedy set selected at step t,
$U_t \neq \varnothing \implies |S_{i_t} \cap U_t| \geq 1$,
$U_{t+1} := U_t \setminus S_{i_t}$, $|U_{t+1}| = |U_t| - |S_{i_t} \cap U_t| \leq |U_t| - 1$
$\implies \exists r \leq |U|:\ U_r = \varnothing$.

2. Feasibility: $\mathcal{A}(I)$ is feasible $\iff$ $\mathcal{A}(I) \subseteq S \ \wedge \bigcup_{S_j \in \mathcal{A}(I)} S_j = U$.

$T_t := \{S_{i_s} : 0 \leq s < t\}$, $T_0 = \varnothing$, $T_{t+1} = T_t \cup \{S_{i_t}\}$,
$T_t \subseteq S \ \wedge\ U_t = U \setminus \bigcup_{S_j \in T_t} S_j$ (induction on t),
$\mathcal{A}(I) = T_r \subseteq S \ \wedge\ \varnothing = U_r = U \setminus \bigcup_{S_j \in T_r} S_j \implies \bigcup_{S_j \in \mathcal{A}(I)} S_j = U$.

3. Efficiency: $\mathcal{A}$ is efficient $\iff$ $\exists q$ polynomial $\forall I \in \mathcal{I}:\ \mathrm{time}_{\mathcal{A}}(I) \leq q(|I|)$.

operations per call $\in O(|S|\,|U|)$, # recursive calls $= r + 1 \leq |U| + 1$,
$|U| \leq |I| \ \wedge\ |S| \leq |I| \implies$ # operations $\in O(|S|\,|U|^2) \subseteq O(|I|^3)$,
$\mathrm{time}_{\text{one operation}} \in \mathrm{poly}(|I|) \implies \mathrm{time}_{\mathcal{A}}(I) \in \mathrm{poly}(|I|)$.

□

Definition 1.14 (Harmonic number)**.**

$$H_0 := 0, \qquad H_n := \sum_{\ell=1}^{n} \frac{1}{\ell} \qquad (n \in \mathbb{N}_{\geq 1}).$$

$H_n:$ the n-th harmonic number.

Lemma 1.15 (Price of the first greedy set)**.**

$$I = (U, S, w) \in \mathcal{I}, \qquad n := |U|,$$
$$\mathrm{OPT}^* := \mathrm{OPT}(I), \qquad \mathrm{OPT} := \mathrm{OPT}^* \setminus \{\varnothing\}, \qquad c(\mathrm{OPT}) = c(\mathrm{OPT}^*),$$
$$S_i \in \arg\min_{S_j \in S} \mathrm{ppi}(S_j) \quad \Longrightarrow \quad \mathrm{ppi}(S_i) \leq \frac{c(\mathrm{OPT})}{n}.$$

Proof. Idea.

$$\text{loss of greedy pick at } i \leq \text{loss of minimum optimal pick at } i.$$

$$\bigcup_{O_j \in \mathrm{OPT}} O_j = U \implies \sum_{O_j \in \mathrm{OPT}} |O_j| \geq n.$$
$$c(\mathrm{OPT}) = \sum_{O_j \in \mathrm{OPT}} w(O_j) = \sum_{O_j \in \mathrm{OPT}} |O_j| \,\mathrm{ppi}(O_j).$$
$$\mathrm{ppi}(S_i) = \min_{S_j \in S} \mathrm{ppi}(S_j) \leq \min_{O_j \in \mathrm{OPT}} \mathrm{ppi}(O_j) \leq \frac{\sum_{O_j \in \mathrm{OPT}} |O_j| \,\mathrm{ppi}(O_j)}{\sum_{O_j \in \mathrm{OPT}} |O_j|} = \frac{c(\mathrm{OPT})}{\sum_{O_j \in \mathrm{OPT}} |O_j|} \leq \frac{c(\mathrm{OPT})}{n}.$$

□

Remark (Price per item as average loss)**.**

$$\mathrm{ppi}(S_i) = \underbrace{\frac{w(S_i)}{|S_i|}}_{\text{average loss (cost paid) per newly covered element}}, \qquad |S_i| > 0.$$

Theorem 1.16 (Approximation ratio of greedy set cover)**.**

GreedySetCover *is an* H_n*-approximation algorithm.*

$$n := |U|.$$

Proof. GreedySetCover is an H_n-approximation algorithm $\overset{\text{Def. 1.7, Thm. 1.13}}{\iff}$ $\forall I \in \mathcal{I}$: $c\big(\mathcal{A}(I)\big) \le H_n\, c\big(\mathrm{OPT}(I)\big)$.

Fix arbitrary $I = (U, S, w) \in \mathcal{I}$.

Strong induction on n.

1. Base case: $n = 1 \implies \forall S_j \in S :\ S_j \neq \varnothing \implies S_j = U$,
$c\big(\mathcal{A}(I)\big) = \min\limits_{S_j \in S:\ S_j \neq \varnothing} w(S_j) = c\big(\mathrm{OPT}(I)\big) = H_1\, c\big(\mathrm{OPT}(I)\big)$.

2. Inductive step: $n \ge 2$.

$S_i \in \arg\min\limits_{S_j \in S} \mathrm{ppi}(S_j)$ (first greedy set),

$k := |S_i| \ge 1, \qquad U' := U \setminus S_i, \qquad |U'| = n - k,$
$S' := \{\, S'_j := S_j \setminus S_i : S_j \in S \,\}, \qquad w(S'_j) := w(S_j), \qquad I' := (U', S', w),$
$C' := c\big(\mathsf{GreedySetCover}(U', S', w)\big)$.

$$\begin{aligned}
c\big(\mathcal{A}(I)\big) &= w(S_i) + C' \\
&\overset{\text{Def. of ppi}}{=} k\ \mathrm{ppi}(S_i) + C' \\
&\le k\ \mathrm{ppi}(S_i) + H_{n-k} c(\mathrm{OPT}) \\
&\overset{\text{Lemma 1.15}}{\le} k\,\frac{c(\mathrm{OPT})}{n} + H_{n-k} c(\mathrm{OPT}) = \left(\frac{k}{n} + H_{n-k}\right) c(\mathrm{OPT}) \le H_n c(\mathrm{OPT}).
\end{aligned}$$

$n = k \implies C' = 0$,
$n > k : \quad \underbrace{C' \le H_{n-k} c(\mathrm{OPT}(I'))}_{\text{IH}} \le H_{n-k} c(\mathrm{OPT})$.

$$\frac{k}{n} = \sum_{\ell=n-k+1}^{n} \frac{1}{n} \le \sum_{\ell=n-k+1}^{n} \frac{1}{\ell} = H_n - H_{n-k}.$$

IH : induction hypothesis.

□

Example 1.17 (Tight bound for greedy set cover)**.**

$k \in \mathbb{N}_{\geq 2}, \qquad \forall i \in \{1, \dots, k\}: \quad |A_i| = |B_i| = 2^{i-1}, \qquad S_i := A_i \cup B_i,$

$\forall i: \ A_i \cap B_i = \varnothing, \qquad \forall i \neq j: \ S_i \cap S_j = \varnothing,$

$$U := \bigcup_{i=1}^{k} S_i, \qquad n := |U| = \sum_{i=1}^{k} 2^i = 2(2^k - 1).$$

$$S_{k+1} := \bigcup_{i=1}^{k} A_i, \qquad S_{k+2} := \bigcup_{i=1}^{k} B_i,$$

$S_{k+1} \cap S_{k+2} = \varnothing \ \wedge \ S_{k+1} \cup S_{k+2} = U,$

$S := \{S_1, \dots, S_{k+2}\}, \qquad \forall S_i \in S: \ w(S_i) := 1, \qquad I_k := (U, S, w).$

$$j \in \{1, \dots, k\}, \qquad U_j := \bigcup_{i=1}^{j} S_i, \qquad |U_j| = 2(2^j - 1),$$

$$|S_j \cap U_j| = 2^j, \qquad |S_{k+1} \cap U_j| = |S_{k+2} \cap U_j| = \sum_{i=1}^{j} 2^{i-1} = 2^j - 1,$$

$$\forall i \leq j: \quad \operatorname{ppi}(S_j \cap U_j) = \frac{1}{2^j} \leq \frac{1}{2^i} = \operatorname{ppi}(S_i \cap U_j),$$

$$\operatorname{ppi}(S_j \cap U_j) = \frac{1}{2^j} < \frac{1}{2^j - 1} = \operatorname{ppi}(S_{k+1} \cap U_j) = \operatorname{ppi}(S_{k+2} \cap U_j),$$

$\forall i > j, \ i \leq k: \quad S_i \cap U_j = \varnothing \implies \operatorname{ppi}(S_i \cap U_j) = \infty.$

$S_k \longrightarrow S_{k-1} \longrightarrow \cdots \longrightarrow S_1,$

$\mathcal{A}(I_k) = \{S_1, \dots, S_k\}, \qquad c\big(\mathcal{A}(I_k)\big) = k.$

$\max\limits_{S_i \in S} |S_i| = 2^k < 2(2^k - 1) = n \implies c\big(\operatorname{OPT}(I_k)\big) \geq 2,$

$S_{k+1} \cup S_{k+2} = U \ \wedge \ c(\{S_{k+1}, S_{k+2}\}) = 2 \implies c\big(\operatorname{OPT}(I_k)\big) = 2.$

$$\frac{c\big(\mathcal{A}(I_k)\big)}{c\big(\operatorname{OPT}(I_k)\big)} = \frac{k}{2} = \frac{1}{2} \log_2\Big(\frac{n}{2} + 1\Big) = \Theta(\log n) = \Theta(H_n).$$

Theorem 1.18 (Hardness of approximating set cover (Dinur–Steurer, 2014))**.**

$\forall \varepsilon \in (0, 1): \quad (1 - \varepsilon) \ln |U|$*-approximation of* $\Pi_{\text{minimum set cover}}$ *is NP-hard.*

Proof. □

Questions

1. Should Definition 1.1 explicitly define $\alpha : \mathcal{I} \to \mathbb{R}_{\geq 1}$ and use $\alpha(I)$ in the approximation inequalities, allowing instance-dependent factors such as $\alpha(I) = H_{|U|}$ and constant factors as constant functions $\alpha(I) = a$ for all I?
2. Should an α-approximation algorithm be required to be efficient, i.e., run in polynomial time? The introduction of the lecture notes describes approximation algorithms as efficient, but Definition 1.1 only states the approximation inequality.
3. Should the minimum set cover problem require $|U| \geq 1$ so that the H_n-approximation guarantee always has $H_n \geq 1$? The proof of Theorem 1.3 in the lecture notes uses $n = 0$ as its base case, but $H_0 = 0$, whereas approximation factors are normally required to satisfy $\alpha \geq 1$. If the empty universe is allowed, how should the approximation factor be stated? Does Definition 1.1 allow α to depend on n, with $\alpha(0) = 1$ and $\alpha(n) = H_n$ for $n \geq 1$, or

should the empty case be handled separately from the approximation guarantee?

2 Approximation schemes

Definition 2.1 (Polynomial time approximation scheme (PTAS))**.**

$\mathcal{A}$ is a PTAS for $\Pi_{\text{optimization}} \iff$

$\forall \varepsilon > 0\ \exists q_\varepsilon$ polynomial $\forall I \in \mathcal{I}$:

$\mathcal{A}_\varepsilon(I)$ terminates and is feasible for $I \wedge$

$$\begin{cases} c\big(\mathcal{A}_\varepsilon(I)\big) \leq (1+\varepsilon) c\big(\mathrm{OPT}(I)\big), & \text{minimization,} \\ c\big(\mathcal{A}_\varepsilon(I)\big) \geq (1-\varepsilon) c\big(\mathrm{OPT}(I)\big), & \text{maximization} \end{cases} \wedge$$

$\mathrm{time}_{\mathcal{A}_\varepsilon}(I) \leq q_\varepsilon(|I|)$.

$$\Pi_{\text{optimization}} := (\mathcal{I}, \mathcal{S}), \qquad \mathrm{OPT}(I) \in \mathcal{S}_I.$$
$$\forall \varepsilon > 0 : \quad \mathcal{A}_\varepsilon : \mathcal{I} \to \mathcal{S}.$$

ε : approximation parameter (fixed),
$\mathcal{A}_\varepsilon$: algorithm for fixed ε,
q_ε : polynomial in $|I|$ whose coefficients and degree may depend on ε.

Definition 2.2 (Fully polynomial time approximation scheme (FPTAS))**.**

$\mathcal{A}$ is an FPTAS for $\Pi_{\text{optimization}} \iff$

$\exists q$ polynomial $\forall \varepsilon > 0\ \forall I \in \mathcal{I}$:

$\mathcal{A}(I, \varepsilon)$ terminates and is feasible for $I \wedge$

$$\begin{cases} c\big(\mathcal{A}(I,\varepsilon)\big) \leq (1+\varepsilon) c\big(\mathrm{OPT}(I)\big), & \text{minimization,} \\ c\big(\mathcal{A}(I,\varepsilon)\big) \geq (1-\varepsilon) c\big(\mathrm{OPT}(I)\big), & \text{maximization} \end{cases} \wedge$$

$$\mathrm{time}_{\mathcal{A}}(I, \varepsilon) \leq q\left(|I|, \frac{1}{\varepsilon}\right).$$

$$\Pi_{\text{optimization}} := (\mathcal{I}, \mathcal{S}), \qquad \mathrm{OPT}(I) \in \mathcal{S}_I.$$
$$\mathcal{A} : \mathcal{I} \times \mathbb{R}_{>0} \to \mathcal{S}.$$

ε : approximation parameter (input),
q : one fixed polynomial in $|I|$ and $1/\varepsilon$.

Example 2.3 (PTAS versus FPTAS running time)**.**

$\mathrm{time}_{\mathcal{A}_\varepsilon}(I) \in O\big(|I|^{1/\varepsilon}\big)$.

$\varepsilon > 0$ fixed : $\quad |I|^{1/\varepsilon} \in \mathrm{poly}(|I|) \quad \Longrightarrow \quad$ PTAS running-time bound.

$|I| = 2$: $\quad |I|^{1/\varepsilon} = 2^{1/\varepsilon} \notin \mathrm{poly}(1/\varepsilon) \quad \Longrightarrow \quad$ not an FPTAS running-time bound.

2.1 Knapsack

Definition 2.4 (Knapsack problem)**.**

$$\Pi_{\text{Knapsack}} = \big(I,\ \text{OPT}(I)\big).$$

$$\begin{aligned} \text{Input:} \quad & I = \{S, (s_i, p_i)_{i\in S}, B\}, \qquad |S| = n, \\ & \forall i \in S:\ s_i, p_i \in \mathbb{Z}^+. \end{aligned}$$

$$\text{Output:} \quad \text{OPT}(I) \in \mathcal{S}_I := \underset{\substack{T\subseteq S \\ \sum_{i\in T} s_i \le B}}{\arg\max} \sum_{i\in T} p_i.$$

$$\begin{aligned} I &: \text{ knapsack instance,} \\ S &: \text{ finite set of available items,} \\ n &: \text{ number of items,} \\ s_i &: \text{ size of item } i, \\ p_i &: \text{ profit of item } i, \\ B &: \text{ budget (capacity),} \\ T &: \text{ selected subset of items,} \\ \mathcal{S}_I &: \text{ feasible subsets with maximum total profit,} \\ \text{OPT}(I) &: \text{ optimal subset returned for } I. \end{aligned}$$

$$\sum_{i\in T} s_i \le B \quad \Longrightarrow \quad T \subseteq \{i \in S : s_i \le B\},$$

$$S \leftarrow \{i \in S : s_i \le B\} \quad \Longrightarrow \quad \text{W.l.o.g. } \forall i \in S:\ s_i \le B.$$

Example 2.5 (Knapsack)**.**

$$I = \{S = \{1 : \langle 10, 130\rangle,\ 2 : \langle 7, 103\rangle,\ 3 : \langle 6, 91\rangle,\ 4 : \langle 4, 40\rangle,\ 5 : \langle 3, 38\rangle\},\ B = 10\}.$$

$$\begin{aligned} &\text{OPT}(I) = \{2, 5\} \in \mathcal{S}_I, \\ &\sum_{i\in \text{OPT}(I)} s_i = 7 + 3 = 10 \le B, \\ &\sum_{i\in \text{OPT}(I)} p_i = 103 + 38 = 141. \end{aligned}$$

Definition 2.6 (Exact dynamic programming algorithm)**.**

1. For each item i, the decision is

$$i \in \text{OPT}(I) \quad \text{(take)} \qquad \text{or} \qquad i \notin \text{OPT}(I) \quad \text{(skip)}.$$

2. Two possible subproblem formulations:

$$F(i, b) := \max_{\substack{T\subseteq\{1,\dots,i\} \\ \sum_{j\in T} s_j \le b}} \sum_{j\in T} p_j,$$

$$M[i, p] := \min_{\substack{T\subseteq\{1,\dots,i\} \\ \sum_{j\in T} p_j \ge p}} \sum_{j\in T} s_j, \qquad \min \varnothing := \infty.$$

3. Base cases and recurrences:

$$\forall b \geq 0 : \; F(0,b) = 0, \qquad \forall i \in \{0,\dots,n\} : \; F(i,0) = 0,$$

$$\forall i \in \{1,\dots,n\} \; \forall b > 0 : \quad F(i,b) = \begin{cases} F(i-1,b), & s_i > b, \\ \max\{\underbrace{F(i-1,b)}_{\text{skip}}, \; \underbrace{p_i + F(i-1,b-s_i)}_{\text{take}}\}, & s_i \leq b. \end{cases}$$

$$\forall p > 0 : \; M[0,p] = \infty,$$

$$\forall i \in \{0,\dots,n\} \; \forall p \leq 0 : \; M[i,p] = 0,$$

$$\forall i \in \{1,\dots,n\} \; \forall p > 0 : \quad M[i,p] = \begin{cases} \min\{M[i-1,p], s_i\}, & p \leq p_i, \\ \min\{\underbrace{M[i-1,p]}_{\text{skip}}, \; \underbrace{s_i + M[i-1,p-p_i]}_{\text{take}}\}, & p > p_i. \end{cases}$$

4. Recover the maximum profit:

$$p_{\max} := \max\big(\{0\} \cup \{p_i : i \in S\}\big), \qquad p^* := \max\{p \in \{0,\dots,np_{\max}\} : M[n,p] \leq B\}.$$

$$\sum_{j \in \mathrm{OPT}(I)} p_j = F(n,B) = p^*.$$

5. Backtracking:

$$F : \quad (i,b,T) \leftarrow (n,B,\varnothing),$$

$$\text{while } i > 0 : \quad (i,b,T) \leftarrow \begin{cases} (i-1,b,T), & F(i,b) = F(i-1,b), \\ (i-1,b-s_i,T \cup \{i\}), & F(i,b) > F(i-1,b). \end{cases}$$

$$M : \quad (i,p,T) \leftarrow (n,p^*,\varnothing),$$

$$\text{while } i > 0 : \quad (i,p,T) \leftarrow \begin{cases} (i-1,p,T), & M[i,p] = M[i-1,p], \\ (i-1,\max\{0,p-p_i\},T \cup \{i\}), & M[i,p] < M[i-1,p]. \end{cases}$$

$$\text{return } T \in \mathcal{S}_I.$$

6. Running time:

$$\begin{aligned} F : \; & O(n(\lfloor B \rfloor + 1)), \\ M : \; & O(n(np_{\max} + 1)) = O(n^2 p_{\max}) \quad (S \neq \varnothing), \\ \text{backtracking} : \; & O(n). \end{aligned}$$

i : number of available items after relabeling $S = \{1,\dots,n\}$,
b : capacity,
p : target profit,
$F(i,b)$: maximum profit with capacity b using the first i items,
$M[i,p]$: minimum size for profit at least p using the first i items,
$p_{\max}$: maximum item profit (zero if $S = \varnothing$),
p^* : largest profit whose minimum required size is at most B,
T : selected items recovered by backtracking (skip on ties).

Remark (Pseudo-polynomial running time)**.**

$$\begin{aligned} \text{bits}(B) = \Theta(\log B), \qquad & \text{bits}(p_{\max}) = \Theta(\log p_{\max}), \\ B = 2^k \implies & O(nB) = O(n2^k), \\ p_{\max} = 2^k \implies & O(n^2 p_{\max}) = O(n^2 2^k). \end{aligned}$$

$$\begin{matrix} O(nB) \subseteq \text{poly}(n, B), \\ O(n^2 p_{\max}) \subseteq \text{poly}(n, p_{\max}) \end{matrix} \implies \text{pseudo-polynomial.}$$

Definition 2.7 (FPTAS-Knapsack)**.**

$$\text{Input:} \quad I = \big(S, (s_i, p_i)_{i \in S}, B\big), \qquad \varepsilon > 0.$$

$$\text{Output:} \quad T \subseteq S, \qquad \sum_{i \in T} s_i \le B.$$

1. If $S = \varnothing$, return $\varnothing$. Otherwise, set

$$n := |S|, \qquad p_{\max} := \max_{i \in S} p_i, \qquad k := \max\left\{1, \left\lfloor \frac{\varepsilon p_{\max}}{n} \right\rfloor\right\}.$$

2. Round down the profits:

$$\forall i \in S: \quad p'_i \leftarrow k \left\lfloor \frac{p_i}{k} \right\rfloor.$$

3. Run the exact knapsack algorithm from Definition 2.6 with

$$I' = \big(S, (s_i, p'_i)_{i \in S}, B\big), \qquad \text{DP columns: } 0, k, 2k, \ldots$$

and return its selected items.

ε : approximation parameter,
k : rounding step (see the approximation remark below),
p'_i : rounded profit of item i,
I' : rounded instance,
T : selected items, evaluated using their original profits p_i.

Remark (Deriving the FPTAS and choosing k)**.**

$$T = \mathcal{A}(I, \varepsilon), \qquad \forall \varepsilon > 0 \ \forall I:$$

$$\text{1. Feasibility:} \quad T \subseteq S, \qquad \sum_{i \in T} s_i \le B.$$

$$\text{2. Approximation:} \quad p(\text{OPT}'(I)) \ge (1 - \varepsilon) p(\text{OPT}(I)).$$

$$\text{3. Running time:} \quad \text{time}_{\mathcal{A}}(I, \varepsilon) \le q(|I|, 1/\varepsilon), \qquad q \text{ polynomial.}$$

1. Feasibility:

$$S, s_i, B \text{ unchanged}, \qquad p_i \to p'_i \implies T \subseteq S, \qquad \sum_{i \in T} s_i \le B.$$

2. Approximation:

$$\underbrace{\text{loss}(\text{OPT}(I)) \le \varepsilon p(\text{OPT}(I))}_{\text{sufficient loss bound}}$$

$$\begin{aligned} &\iff p(\text{OPT}(I)) - p'(\text{OPT}(I)) \le \varepsilon p(\text{OPT}(I)) \\ &\iff p'(\text{OPT}(I)) \ge p(\text{OPT}(I)) - \varepsilon p(\text{OPT}(I)) \\ &\iff p(\text{OPT}(I)) \ge p(\text{OPT}'(I)) \ge p'(\text{OPT}'(I)) \ge p'(\text{OPT}(I)) \ \ge \ (1 - \varepsilon) p(\text{OPT}(I)). \end{aligned}$$

$$p(T) := \sum_{i \in T} p_i, \qquad p'(T) := \sum_{i \in T} p'_i, \qquad \text{loss}(T) := p(T) - p'(T).$$

$$\forall i \in S: \quad 0 \le p_i - p'_i < k,$$

$$\text{loss}(\text{OPT}(I)) = \sum_{i \in \text{OPT}(I)} (p_i - p'_i) \le nk,$$

$$p_{\max} \le p(\text{OPT}(I)) \implies \varepsilon p_{\max} \le \varepsilon\, p(\text{OPT}(I)).$$

$$\text{loss}(\text{OPT}(I)) \le nk \xRightarrow{\text{sufficient loss bound}} \text{loss}(\text{OPT}(I)) \le nk \le \varepsilon p_{\max} \le \varepsilon\, p(\text{OPT}(I)).$$

$$nk \le \varepsilon p_{\max} \iff k \le \frac{\varepsilon p_{\max}}{n} \xRightarrow{\text{largest integer } k} k = \left\lfloor \frac{\varepsilon p_{\max}}{n} \right\rfloor.$$

$$k := \max\left\{1, \left\lfloor \frac{\varepsilon p_{\max}}{n} \right\rfloor\right\}, \qquad k = 1 \implies p'_i = p_i \implies \text{loss}(T) = 0.$$

3. Fully polynomial running time:

$$\frac{p_{\max}}{k} \le \frac{2n}{\varepsilon},$$

$$\text{time}_{\mathcal{A}}(I, \varepsilon) \in O\left(n\left(\left\lfloor \frac{np_{\max}}{k} \right\rfloor + 1\right)\right) \subseteq O\left(n + \frac{n^3}{\varepsilon}\right) \subseteq \text{poly}(|I|, 1/\varepsilon).$$

T : subset of items,
$\text{OPT}'(I)$: optimal selected subset for the rounded instance I',
$p(T)$: original total profit of T,
$p'(T)$: rounded total profit of T,
$\text{loss}(T)$: profit lost through rounding,
n : number of remaining items ($n > 0$).

Example 2.8 (FPTAS-Knapsack)**.**

$$I = \{S = \{1 : \langle 10, 130\rangle,\ 2 : \langle 7, 103\rangle,\ 3 : \langle 6, 91\rangle,\ 4 : \langle 4, 40\rangle,\ 5 : \langle 3, 38\rangle\},\ B = 10\}.$$

$$\varepsilon = \frac{1}{2}, \qquad n = 5, \qquad p_{\max} = 130, \qquad k = \max\left\{1, \left\lfloor \frac{(1/2) \cdot 130}{5} \right\rfloor\right\} = 13.$$

$$I' = \{S = \{1 : \langle 10, 130\rangle,\ 2 : \langle 7, 91\rangle,\ 3 : \langle 6, 91\rangle,\ 4 : \langle 4, 39\rangle,\ 5 : \langle 3, 26\rangle\},\ B = 10\}.$$

$$\text{OPT}(I) = \{2, 5\}, \qquad p(\text{OPT}(I)) = 103 + 38 = 141,$$
$$p'(\text{OPT}(I)) = 91 + 26 = 117,$$
$$\text{OPT}'(I) = \{3, 4\} \quad \text{(lecture's tie choice)}, \quad p'(\text{OPT}'(I)) = 91 + 39 = 130,$$
$$p(\text{OPT}'(I)) = 91 + 40 = 131.$$

$$\text{loss}(\text{OPT}(I)) = 141 - 117 = 24, \qquad \text{loss}(\text{OPT}'(I)) = 131 - 130 = 1.$$

$$p(\text{OPT}'(I)) = 131 \ge (1 - \varepsilon) p(\text{OPT}(I)) = \frac{1}{2} \cdot 141 = 70.5.$$

$$p'(\{1\}) = p'(\{3, 4\}) = 130, \qquad \text{skip on ties} \implies \text{OPT}'(I) = \{1\}, \quad p(\{1\}) = 130.$$

Definition 2.9 (1-Lipschitz continuity)**.**

$$f : D \subseteq \mathbb{R} \to \mathbb{R}, \qquad f \text{ is 1-Lipschitz continuous} \iff \forall x, y \in D: \ |f(x) - f(y)| \le |x - y|.$$

D : domain,
x, y : input values,
$f(x), f(y)$: corresponding output values.

Remark (Why round profits instead of sizes?). 1. Profits: fix B, all sizes, and all profits except p_i.

$$f(x) := p_{\mathrm{OPT}}(p_1, \ldots, p_{i-1}, x, p_{i+1}, \ldots, p_n),$$
$$\forall \varepsilon > 0: \quad f(p_i) \le f(p_i + \varepsilon) \le f(p_i) + \varepsilon,$$
$$|f(p_i + \varepsilon) - f(p_i)| \le \varepsilon \quad \Longrightarrow \quad f \text{ is 1-Lipschitz continuous.}$$

2. Sizes: one item with profit $P > 0$ and size x.

$$g(x) := \begin{cases} P, & 0 < x \le B, \\ 0, & x > B. \end{cases}$$

$$\forall \varepsilon > 0: \quad |g(B + \varepsilon) - g(B)| = P, \qquad \lim_{\varepsilon \downarrow 0} g(B + \varepsilon) = 0 \neq g(B) = P.$$

$$\varepsilon \to 0, \quad P \text{ arbitrarily large} \quad \Longrightarrow \quad g \text{ is not continuous at } B.$$

p_{OPT} : optimal total profit,
$f(x)$: optimal total profit when item i's profit is x,
$g(x)$: optimal total profit when the single item's size is x,
ε : positive change in one profit or size (real-valued extension),
P : fixed profit of the single item.

2.2 Bin packing

Definition 2.10 (Bin packing problem).

$$\Pi_{\mathrm{BinPacking}} = \big(I,\ \mathrm{OPT}(I)\big).$$

$$\text{Input:} \quad I = \{S, (\mathrm{size}(i))_{i \in S}\}, \qquad |S| = n,$$
$$\forall i \in S: \quad \mathrm{size}(i) \in (0, 1].$$

$$\text{Output:} \quad \mathrm{OPT}(I) \in \mathcal{S}_I := \operatorname*{arg\,min}_{\substack{\mathcal{B} \text{ partitions } S \\ \forall b \in \mathcal{B}:\ \sum_{i \in b} \mathrm{size}(i) \le 1}} |\mathcal{B}|.$$

I : bin-packing instance,
S : finite set of items,
n : number of items,
$\mathrm{size}(i)$: size of item i,
b : items assigned to one unit-capacity bin,
$\mathcal{B}$: collection of bins, assigning each item exactly once,
$|\mathcal{B}|$: number of bins,
$\mathcal{S}_I$: feasible bin assignments using the minimum number of bins,
$\mathrm{OPT}(I)$: optimal bin assignment.

Remark.

$$\mathrm{size}(I) := \sum_{i=1}^{n} \mathrm{size}(i) \le |\,\mathrm{OPT}(I)|.$$

Example 2.11 (Bin packing)**.**

$$I = \{S = \{1 : 0.5,\ 2 : 0.1,\ 3 : 0.1,\ 4 : 0.1,\ 5 : 0.5,\ 6 : 0.4,\ 7 : 0.5,\ 8 : 0.4,\ 9 : 0.4\}\}.$$

$$\text{size}(I) = 3 \cdot 0.5 + 3 \cdot 0.4 + 3 \cdot 0.1 = 3 \quad \Longrightarrow \quad |\,\text{OPT}(I)| \geq 3.$$

$$b_1 = \{1, 6, 2\}, \qquad b_2 = \{5, 8, 3\}, \qquad b_3 = \{7, 9, 4\}.$$

$$\forall j \in \{1, 2, 3\} : \quad \sum_{i \in b_j} \text{size}(i) = 0.5 + 0.4 + 0.1 = 1.$$

$$\text{OPT}(I) = \{b_1, b_2, b_3\}, \qquad |\,\text{OPT}(I)| = 3.$$

Definition 2.12 (FirstFit algorithm)**.**

$$\begin{aligned} \text{Input:} \quad & I = \{S = \{1, \ldots, n\},\ (\text{size}(i))_{i \in S}\}, \\ & \forall i \in S :\ \text{size}(i) \in (0, 1]. \\ \text{Output:} \quad & B = \{b_1, \ldots, b_m\}. \end{aligned}$$

$$\text{size}(b) := \sum_{i \in b} \text{size}(i), \qquad \text{free}(b) := 1 - \text{size}(b).$$

Algorithm 2 $\mathsf{FirstFit}(I)$

```
1: m ← 0
2: for i = 1, ..., n do
3:     if ∃ℓ ∈ {1, ..., m} : free(b_ℓ) ≥ size(i) then
4:         j ← min{ℓ ∈ {1, ..., m} : free(b_ℓ) ≥ size(i)}
5:         b_j ← b_j ∪ {i}
6:     else
7:         m ← m + 1
8:         b_m ← {i}
9:     end if
10: end for
11: return B = {b_1, ..., b_m}
```

B : collection of bins,
m : number of bins created,
b_j : j-th bin in creation order,
$\text{size}(b)$: total size of items in bin b,
$\text{free}(b)$: remaining capacity of bin b,
j : index of the first existing bin that fits item i.

Lemma 2.13 (FirstFit: at most one bin at most half full)**.**

$$B = \mathsf{FirstFit}(I) \quad \Longrightarrow \quad \left|\left\{b \in B : \text{size}(b) \leq \frac{1}{2}\right\}\right| \leq 1.$$

Proof. By contradiction.

$$\exists j < \ell : \quad \text{size}(b_j) \leq \frac{1}{2}, \qquad \text{size}(b_\ell) \leq \frac{1}{2}.$$

For the item i opening b_ℓ:

$$\underbrace{\text{free}(b_j)}_{\text{when } b_\ell \text{ opens}} \geq 1 - \text{size}(b_j) \geq \frac{1}{2} \geq \text{size}(b_\ell) \geq \text{size}(i).$$

$$i \text{ fits in } b_j \quad \Longrightarrow \quad b_\ell \text{ is not opened} \quad \Longrightarrow \quad \bot.$$

□

Theorem 2.14 (FirstFit is a 2-approximation)**.**

$$\begin{aligned} \forall I: \quad & |\mathsf{FirstFit}(I)| \leq 2|\operatorname{OPT}(I)|, \\ & \text{time}_{\mathsf{FirstFit}}(I) \in \text{poly}(|I|). \end{aligned}$$

Proof.

$$B := \mathsf{FirstFit}(I), \qquad m := |B|.$$

$$S = \varnothing \quad \Longrightarrow \quad m = 0 = |\operatorname{OPT}(I)|.$$

For $S \neq \varnothing$, by Lemma 2.13:

$$\frac{m-1}{2} < \sum_{b \in B} \text{size}(b) = \text{size}(I) \leq |\operatorname{OPT}(I)|.$$

$$m - 1 < 2|\operatorname{OPT}(I)|, \qquad m, |\operatorname{OPT}(I)| \in \{0\} \cup \mathbb{Z}^+ \quad \Longrightarrow \quad m \leq 2|\operatorname{OPT}(I)|.$$

$$\text{time}_{\mathsf{FirstFit}}(I) \in O\left(\sum_{i=1}^{n} i\right) = O(n^2) \subseteq \text{poly}(|I|).$$

□

Example 2.15 (FirstFit)**.**

$$I = \{S = \{1:0.5,\ 2:0.1,\ 3:0.1,\ 4:0.1,\ 5:0.5,\ 6:0.4,\ 7:0.5,\ 8:0.4,\ 9:0.4\}\}.$$

$$\begin{aligned} b_1 &= \{1,2,3,4\}, & \text{size}(b_1) &= 0.5 + 0.1 + 0.1 + 0.1 = 0.8, \\ b_2 &= \{5,6\}, & \text{size}(b_2) &= 0.5 + 0.4 = 0.9, \\ b_3 &= \{7,8\}, & \text{size}(b_3) &= 0.5 + 0.4 = 0.9, \\ b_4 &= \{9\}, & \text{size}(b_4) &= 0.4. \end{aligned}$$

$$\mathsf{FirstFit}(I) = \{b_1, b_2, b_3, b_4\}, \qquad |\mathsf{FirstFit}(I)| = 4 \leq 2|\operatorname{OPT}(I)| = 6.$$

Definition 2.16 (SortedFirstFit algorithm)**.**

$$\begin{aligned} \text{Input:} \quad & I = \{S = \{1, \ldots, n\},\ (\text{size}(i))_{i \in S}\}, \\ & \forall i \in S: \quad \text{size}(i) \in (0, 1]. \\ \text{Output:} \quad & B = \{b_1, \ldots, b_m\}. \end{aligned}$$

Algorithm 3 $\mathsf{SortedFirstFit}(I)$

1: Sort and relabel items so that $\text{size}(1) \geq \cdots \geq \text{size}(n)$.
2: **return** $\mathsf{FirstFit}(I)$

B : collection of bins returned by FirstFit after sorting,
m : number of bins,
b_j : j-th bin in creation order.

Theorem 2.17 (SortedFirstFit is a $\frac{3}{2}$-approximation)**.**

$$\forall I: \quad |\mathsf{SortedFirstFit}(I)| \le \frac{3}{2}|\operatorname{OPT}(I)|,$$
$$\operatorname{time}_{\mathsf{SortedFirstFit}}(I) \in \operatorname{poly}(|I|).$$

Proof.

$$k := |\mathsf{SortedFirstFit}(I)|, \qquad r := |\{i \in S : \operatorname{size}(i) > \tfrac{1}{2}\}|.$$
$$k = 1 \implies |\operatorname{OPT}(I)| \ge 1 \implies k = 1 \le \frac{3}{2}|\operatorname{OPT}(I)|.$$

Assume $k \ge 2$.

Case 1: $r \ge \frac{2k}{3}$.

$$|\operatorname{OPT}(I)| \ge r \ge \frac{2k}{3} \implies k \le \frac{3}{2}|\operatorname{OPT}(I)|.$$

Case 2: $r < \frac{2k}{3}$.

$$k_1 := \left\lceil \frac{2k}{3} \right\rceil - 1, \qquad k_2 := k - k_1.$$

The first k_1 bins are early; the rest are late. Late items have size $\le \frac{1}{2}$, so each late bin except possibly the last has at least two items.

$$\#\{\text{late items}\} \ge 2(k_2 - 1) + 1 = 2k - 2\left\lceil \frac{2k}{3} \right\rceil + 1 \ge k_1.$$

Pair distinct late items $i_1, \ldots, i_{k_1}$ with early bins $b_1, \ldots, b_{k_1}$. Each item failed to fit:

$$\forall j \in \{1, \ldots, k_1\}: \quad \operatorname{size}(i_j) + \operatorname{size}(b_j) > 1.$$

$$|\operatorname{OPT}(I)| \ge \operatorname{size}(I) \ge \sum_{j=1}^{k_1} \big(\operatorname{size}(b_j) + \operatorname{size}(i_j)\big)$$
$$> k_1 = \left\lceil \frac{2k}{3} \right\rceil - 1.$$

$$|\operatorname{OPT}(I)| \in \mathbb{Z} \implies |\operatorname{OPT}(I)| \ge \left\lceil \frac{2k}{3} \right\rceil \ge \frac{2k}{3} \implies k \le \frac{3}{2}|\operatorname{OPT}(I)|.$$

$$\operatorname{time}_{\mathsf{SortedFirstFit}}(I) \in O(n \log n) + O(n^2) \subseteq \operatorname{poly}(|I|).$$

□

Definition 2.18 (Partition problem)**.**

$$\Pi_{\text{Partition}} = \big(I,\ \mathsf{Partition}(I)\big).$$

Input: $I = \{x_1, \ldots, x_n\}, \quad S := I, \qquad x_1, \ldots, x_n \in \mathbb{Z}^+.$

Output: $$\mathsf{Partition}(I) = \begin{cases} \text{YES}, & \exists S_1, S_2: \quad S_1 \cup S_2 = S, \quad S_1 \cap S_2 = \varnothing, \\ & \qquad \sum_{x \in S_1} x = \sum_{x \in S_2} x, \\ \text{NO}, & \text{otherwise.} \end{cases}$$

S : multiset of positive integers; repeated occurrences are distinct.

Theorem 2.19 (Hardness of approximation for bin packing)**.**

$$\exists \text{ polynomial-time } \alpha\text{-approximation for bin packing}, \quad 1 \le \alpha < \frac{3}{2} \quad \Longrightarrow \quad P = NP.$$

Proof. Assume

$$\exists \mathcal{A}, \quad 0 < \varepsilon \le \frac{1}{2} : \qquad \forall I' : \quad |\operatorname{OPT}(I')| \le |\mathcal{A}(I')| \le \left(\frac{3}{2} - \varepsilon\right) |\operatorname{OPT}(I')|,$$
$$\operatorname{time}_{\mathcal{A}}(I') \in \operatorname{poly}(|I'|).$$

$$\Pi_{\text{Partition}} = \big(I = \{x_1, \ldots, x_n\},\ \mathsf{Partition}(I)\big), \qquad x_i \in \mathbb{Z}^+.$$

$$X := \sum_{i=1}^{n} x_i.$$

$$\exists i : \ x_i > \frac{X}{2} \quad \Longrightarrow \quad \text{NO}.$$

Otherwise,

$$\Pi_{\text{BinPacking}} = \big(I',\ \operatorname{OPT}(I')\big).$$

$$I' = \left\{\frac{x_1}{X/2}, \ldots, \frac{x_n}{X/2}\right\}.$$

$$\mathsf{Partition}(I) = \text{YES} \iff \exists_{\substack{S_1 \cup S_2 = I \\ S_1 \cap S_2 = \varnothing}} S_1, S_2 : \quad \sum_{x \in S_1} x = \sum_{x \in S_2} x = \frac{X}{2} \iff |\operatorname{OPT}(I')| = 2.$$

$$|\operatorname{OPT}(I')| = 2 \Longrightarrow 2 \le |\mathcal{A}(I')| \le 3 - 2\varepsilon < 3 \Longrightarrow |\mathcal{A}(I')| = 2,$$
$$|\operatorname{OPT}(I')| \ge 3 \Longrightarrow |\mathcal{A}(I')| \ge 3.$$

$$\mathcal{A}_{\text{Partition}}(I) = \begin{cases} \text{YES}, & |\mathcal{A}(I')| = 2, \\ \text{NO}, & |\mathcal{A}(I')| \ge 3. \end{cases}$$

$$\operatorname{time}_{\mathcal{A}_{\text{Partition}}}(I) \in \operatorname{poly}(|I|) \quad \Longrightarrow \quad \Pi_{\text{Partition}} \in P \quad \xRightarrow{\Pi_{\text{Partition}} \text{ NP-hard}} \quad P = NP.$$

□

Questions

1. The introduction to Chapter 2 states that the greedy algorithms from Chapter 1 approximate the optimum within a constant factor, including minimum set cover. Is this a typo? The guarantee for greedy set cover is $H_n = \Theta(\log n)$, where $n = |U|$, so its approximation factor is not constant. Should the statement distinguish this logarithmic factor from the constant factors for matching and vertex cover?
2. For maximization, should the PTAS and FPTAS guarantee be defined as $c(\mathcal{A}(I, \varepsilon)) \ge (1 - \varepsilon)c(\operatorname{OPT}(I))$? How does this relate to Definition 1.1, which uses $c(\operatorname{OPT}(I)) \le \alpha\, c(\mathcal{A}(I))$? Is the intended correspondence $\alpha = 1/(1 - \varepsilon)$ for $0 < \varepsilon < 1$, rather than $\alpha = 1 + \varepsilon$?
3. In Section 2.1, should "the highest profit for an item" be written as "the maximum profit among all items"? Each item has one fixed profit p_i, and $p_{\max} = \max_{i \in S} p_i$ takes the maximum over all items.
4. In the proof of Theorem 2.4, should "$p(S)$ and $'(S)$" read "$p(S)$ and $p'(S)$"? The rounded-profit notation appears to be missing p.

General questions

1. When the lecture notes state a theorem but refer to a paper or another source for its proof, are we expected to study and be able to reproduce that proof, or is it sufficient to know the theorem's statement and how to use it?